

# Auditoria Roadmap para nível corporativo

Os temas que separam um projeto de portfólio de um sistema que uma empresa grande reconheceria como produção — organizados por pilar, prioridade e fase de entrega.

# 1. Como uma empresa grande lê um projeto de portfólio

Um avaliador técnico gasta entre cinco e dez minutos no seu repositório. Nesse tempo ele não lê o código todo — ele procura **sinais**. A tese deste documento é simples: os sinais que importam são poucos, são previsíveis e quase nenhum deles é "mais telas".

São seis perguntas que o avaliador responde, quase sempre nesta ordem:

| A PERGUNTA                   | ONDE ELE PROCURA                                                 | O QUE REPROVA                                                         |
|------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------|
| Isso roda?                   | README, link do deploy, conta de demonstração                    | Precisar clonar e configurar variáveis para ver qualquer coisa        |
| Tem domínio de verdade?      | Nomes de tabelas, tipos, regras de cálculo                       | CRUD genérico com nomes tipo <code>items</code> e <code>status</code> |
| É seguro?                    | Autorização no banco, tratamento de sessão, validação de entrada | Filtro de permissão feito só no front-end                             |
| Dá para confiar em mudanças? | Testes, CI verde, histórico de commits                           | Zero testes ou um único commit "initial commit" com tudo              |
| Foi pensado ou copiado?      | Decisões documentadas, trade-offs assumidos                      | Stack de tutorial sem nenhuma justificativa                           |
| Sabe entregar acabamento?    | Estados de erro e vazio, acessibilidade, responsividade          | Telas que só funcionam no caminho feliz                               |

## Onde o projeto está hoje

A fundação já cobre boa parte do que é *visível*: modelo de domínio tipado e fiel à norma, design system com tokens semânticos, tema claro/escuro, app shell responsivo, painel com indicadores e camada de dados isolada. Isso responde às perguntas 2 e 6.

O que falta é justamente o que costuma decidir a avaliação: **segurança, confiabilidade e prova de que funciona** — perguntas 1, 3, 4 e 5. Os dez pilares a seguir cobrem essa lacuna.

## 2. Os dez pilares

Cada pilar traz o motivo pelo qual ele pesa na avaliação e os itens concretos a implementar. A etiqueta indica a prioridade relativa.

### 01 Arquitetura e modelagem de dados

Crítico

*Sinaliza se você projeta sistemas ou apenas encaixa telas.*

- Schema multi-tenant com `organization_id` em toda tabela de negócio, não como campo opcional.
- Migrations versionadas em `supabase/migrations`, aplicadas por CI — nunca alteração manual no painel.
- Constraints reais no banco: *foreign keys*, `check` de máquina de estados, *unique* nos códigos (AUD-2026-014).
- Índices nas colunas de filtro (`status`, `prazo`, `organization_id`) e explicação do porquê.
- Tipos TypeScript gerados a partir do schema, garantindo que banco e código não diverjam.
- Separação de camadas preservada: componentes nunca falam com o banco direto, só via `lib/queries`.

### 02 Segurança, autenticação e autorização

Crítico

*É o item que mais reprova projeto de portfólio — e o que mais impressiona quando bem feito.*

- **Row Level Security** ativa em todas as tabelas, com políticas por papel (admin, auditor líder, auditor, auditado, gestor).
- Autorização no banco, não no front: mesmo com o token, um auditado não consegue ler auditoria de outra unidade.
- Validação de entrada com *zod* no servidor, não apenas no formulário.
- Trilha de auditoria imutável (`activity_log`) — quem alterou o quê e quando. Em um sistema de auditoria, isso é requisito do próprio domínio.
- *Rate limiting* nas rotas de escrita e nas de autenticação.
- Cabeçalhos de segurança: CSP, `X-Frame-Options`, HSTS.
- Segredos apenas em variáveis de ambiente, com `.env.example` versionado e `.env` nunca.
- Upload de evidências com validação de tipo e tamanho, e URLs assinadas com expiração.

### 03 Testes automatizados

Crítico

*Sem testes, o avaliador assume que qualquer mudança quebra o sistema.*

- **Unitários (Vitest):** cálculo do índice de conformidade ponderado, apuração de prazos vencidos, transições válidas de status.
- **Integração:** as políticas de RLS — o teste mais valioso do projeto. Autentica como cada papel e prova que o acesso indevido falha.
- **E2E (Playwright):** o fluxo que define o produto — criar auditoria, responder checklist, gerar NC, definir plano de ação, verificar eficácia.
- Cobertura publicada com meta declarada (60–70% é honesto e suficiente; 100% não convence ninguém).
- Testes de acessibilidade automatizados com *axe-core* nas telas principais.

### 04 CI/CD e automação

Crítico

*Um badge verde no README responde a pergunta "dá para confiar em mudanças?" antes de qualquer leitura de código.*

- GitHub Actions rodando *lint*, *typecheck*, testes e *build* em cada *pull request*.
- Branch protegida: sem CI verde, não faz *merge*. Mesmo trabalhando sozinho — o hábito é o que se avalia.
- *Preview deploy* automático por PR na Vercel.
- Migrations aplicadas pelo pipeline, não à mão.
- Dependabot ou Renovate para atualização de dependências.
- *Conventional commits* e CHANGELOG gerado — histórico legível conta como documentação.

### 05 Regras de negócio e profundidade de domínio

Alto

*É o que transforma "mais um CRUD" em "um sistema". Também é o que rende assunto na entrevista.*

- **Motor de score:** índice de conformidade ponderado pelo peso de cada requisito, com NC maior derrubando a seção inteira.
- **Máquina de estados** explícita da não-conformidade, com transições proibidas rejeitadas no banco — não se encerra uma NC sem causa raiz registrada.
- **SLA por severidade:** prazo calculado automaticamente conforme a classificação, com escalonamento ao gestor no vencimento.
- **Verificação de eficácia** como etapa obrigatória (cláusula 10.2), impedindo o encerramento prematuro que a própria norma cobra.
- **Relatório de auditoria em PDF** com cabeçalho da organização, constatações e assinaturas — é o entregável real do processo.
- **Programa anual** com visão de calendário e alerta de processos não auditados no ciclo.
- Reincidência: detectar NCs repetidas no mesmo requisito — indicador clássico de ação corretiva ineficaz.

## 06 Experiência de uso e acessibilidade

Alto

*Acabamento é a diferença mais visível entre júnior e pleno.*

- Estados completos em toda tela: carregando (*skeleton*), vazio, erro e sem permissão.
- Execução da auditoria pensada para celular — o auditor preenche em campo, de pé, às vezes sem rede.
- Formulários com validação inline, mensagens específicas e foco no primeiro campo inválido.
- WCAG 2.1 nível AA: contraste, navegação por teclado, *landmarks*, rótulos em ícones.
- Confirmação em ações destrutivas e desfazer onde fizer sentido.
- Busca global com atalho de teclado e filtros persistidos na URL (link compartilhável).
- Otimismo de interface: a ação responde na hora e reverte se o servidor recusar.

## 07 Observabilidade

Médio

*Mostra que você pensa no sistema depois do deploy, não só até ele.*

- Monitoramento de erros com Sentry, incluindo *source maps* e identificação de release.
- Logs estruturados em JSON com *request id* correlacionando cliente e servidor.
- Rota `/api/health` verificando aplicação e banco.
- Core Web Vitals coletados em produção, não apenas medidos no Lighthouse local.
- Página de erro e `not-found` próprias, com caminho de volta.

## 08 Performance

Médio

*Número é argumento. "LCP de 1,2 s" vale mais que "otimizei o front".*

- Server Components como padrão; `"use client"` apenas onde há interatividade real.
- *Streaming* com `loading.tsx` e Suspense nas telas de listagem.
- Paginação e filtro no servidor — nunca trazer a tabela inteira para o navegador.
- Orçamento de bundle vigiado no CI, com falha se estourar o limite.
- Imagens via `next/image` em AVIF/WebP e fontes com `display: swap`.
- Lighthouse acima de 95 em performance, acessibilidade e boas práticas, com o relatório publicado.

## 09 Documentação e apresentação

Alto

*É o pilar de melhor retorno por hora investida — e o mais negligenciado.*

- README com captura de tela logo no topo, link do deploy e credenciais da conta de demonstração.
- Diagrama de arquitetura e diagrama do modelo de dados (Mermaid renderiza direto no GitHub).
- **ADRs** curtas em `docs/adr/`: por que Supabase, por que RLS em vez de checagem na aplicação, por que domínio em português.
- Seed com dados de demonstração realistas e conta somente-leitura para o avaliador entrar sem cadastro.
- Vídeo ou GIF de 60 segundos do fluxo principal.
- `CONTRIBUTING.md` e template de PR — sinaliza que você sabe trabalhar em equipe.

## 10 Conformidade e privacidade

Médio

*Raro em portfólio e altamente valorizado em empresa grande — especialmente num produto de auditoria.*

- Mapeamento LGPD dos dados pessoais tratados e sua base legal.
- Política de retenção e exportação dos dados da organização.
- Anonimização nos dados de demonstração e nos ambientes de teste.
- Registro de consentimento e trilha de acesso a dados sensíveis.

### 3. Roadmap por fases

A ordem é deliberada: primeiro o que sustenta tudo (dados e segurança), depois o que dá profundidade (regras de negócio), por fim o que dá brilho (acabamento e apresentação). Cada fase termina com algo demonstrável.

#### Fase 1 — Fundação de dados e segurança

Crítico

- Projeto Supabase dedicado, com migrations versionadas no repositório.
- Schema completo do domínio, com constraints e índices.
- Autenticação por e-mail e senha, com sessão via middleware.
- RLS ativa em todas as tabelas, com política por papel.
- Tipos TypeScript gerados a partir do schema.
- Reescrita de `lib/queries` apontando para o Supabase — a UI não muda.

**Entregável:** login funcionando e dois usuários de papéis diferentes enxergando conjuntos distintos de dados, com o isolamento provado no banco.

#### Fase 2 — CI e rede de segurança

Crítico

- GitHub Actions com lint, typecheck, testes e build.
- Vitest configurado e as regras de cálculo cobertas por testes.
- Testes de RLS autenticando como cada papel.
- Branch protegida exigindo CI verde.
- Deploy na Vercel com *preview* por PR.

**Entregável:** badge de CI verde no README e link público da aplicação.

#### Fase 3 — O produto de verdade

Alto

- Construtor de checklist com seções, requisitos e pesos.
- Tela de execução da auditoria otimizada para celular, com upload de evidências.
- Motor de score ponderado, recalculado a cada resposta.
- Abertura automática de NC quando a resposta é não conforme.
- Máquina de estados da NC e do plano de ação aplicada no banco.
- SLA por severidade com escalonamento no vencimento.

**Entregável:** ciclo completo percorrível de ponta a ponta, da criação da auditoria ao encerramento da ação corretiva.

## Fase 4 — Entregáveis do processo

Alto

- Relatório de auditoria em PDF com identidade da organização.
- Exportação de indicadores em Excel e CSV.
- Notificações por e-mail de prazo próximo e prazo vencido.
- Programa anual em visão de calendário.
- Indicador de reincidência por requisito.

**Entregável:** o gestor consegue tirar do sistema o documento que hoje ele faria no Word.

## Fase 5 — Acabamento e robustez

Médio

- Estados de carregando, vazio, erro e sem permissão em todas as telas.
- Auditoria de acessibilidade e correção até WCAG AA.
- Testes E2E do fluxo principal com Playwright.
- Sentry, logs estruturados e rota de health check.
- Otimização de performance até Lighthouse acima de 95.

**Entregável:** relatório de Lighthouse e de acessibilidade publicados no repositório.

## Fase 6 — Apresentação

Alto

- README reescrito com capturas, diagramas e conta de demonstração.
- ADRs registrando as decisões e os trade-offs assumidos.
- Seed de demonstração e usuário somente-leitura.
- Vídeo curto do fluxo principal.
- Bullets de currículo derivados das métricas reais do projeto.

**Entregável:** o avaliador entende o projeto em dois minutos sem clonar nada.

## 4. Critério de pronto

Vale como régua para qualquer entrega do projeto. Uma funcionalidade só está concluída quando todos os itens aplicáveis foram atendidos.

### Código

- ☐ Tipado sem `any`, sem *type assertion* desnecessária
- ☐ Lint e typecheck limpos
- ☐ Regra de negócio isolada da camada de UI
- ☐ Sem valor mágico solto — vai para configuração ou constante
- ☐ Nomes no vocabulário do domínio

### Segurança

- ☐ Política de RLS escrita e testada
- ☐ Entrada validada no servidor
- ☐ Nenhum segredo no bundle do cliente
- ☐ Alteração registrada na trilha de auditoria

### Interface

- ☐ Estados de carregando, vazio e erro implementados
- ☐ Funciona a partir de 360 px de largura
- ☐ Navegável apenas pelo teclado
- ☐ Contraste verificado nos dois temas
- ☐ Textos em português consistente com a norma

### Entrega

- ☐ Coberta por teste automatizado
- ☐ CI verde no pull request
- ☐ Commit no padrão *conventional commits*
- ☐ Documentação atualizada quando muda comportamento

### Métricas-alvo do projeto

| MÉTRICA                     | META    | COMO COMPROVAR                             |
|-----------------------------|---------|--------------------------------------------|
| Cobertura de testes         | ≥ 60%   | Relatório publicado pelo CI                |
| Lighthouse — performance    | ≥ 95    | Relatório versionado em <code>docs/</code> |
| Lighthouse — acessibilidade | 100     | Relatório versionado em <code>docs/</code> |
| Largest Contentful Paint    | < 1,5 s | Medição em produção                        |
| Tabelas com RLS ativa       | 100%    | Teste de integração por papel              |
| Erros de tipo               | 0       | <code>tsc --noEmit</code> no CI            |

# 5. Como isso vira currículo

Um projeto só conta no currículo se o recrutador entender o que ele prova. Descreva por **problema resolvido e resultado medido**, nunca por lista de tecnologias. Os bullets abaixo ficam verdadeiros à medida que as fases avançam.

## Bullets de referência

- Desenvolvi um sistema de gestão de auditorias internas segundo a ISO 9001:2015, cobrindo o ciclo completo do programa anual à verificação de eficácia das ações corretivas, em Next.js, TypeScript e PostgreSQL.
- Implementei isolamento multi-tenant com Row Level Security no PostgreSQL, aplicando autorização por papel na camada de dados e comprovando o isolamento por testes de integração.
- Modelei o motor de cálculo do índice de conformidade ponderado por criticidade de requisito, com máquina de estados que impede o encerramento de não-conformidade sem análise de causa registrada.
- Estabeleci pipeline de CI com lint, verificação de tipos, testes e build obrigatórios por pull request, além de preview deploy automático.
- Construí design system próprio com tokens semânticos derivados da classificação de constatações da norma, com tema claro e escuro e conformidade WCAG 2.1 AA.

## O que responder na entrevista

| PERGUNTA PROVÁVEL                                       | O QUE O PROJETO PERMITE RESPONDER                                                                                                     |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Como você garante que um usuário não vê dados de outro? | RLS no PostgreSQL: a política vive no banco, então nem uma falha de front nem uma chamada direta à API contornam o isolamento.        |
| Qual foi a decisão técnica mais difícil?                | Autorizar no banco em vez de na aplicação — custa mais para escrever e depurar, e em troca elimina uma classe inteira de vazamento.   |
| Como você lida com regra de negócio complexa?           | Máquina de estados explícita com transições validadas por constraint, e o cálculo do score isolado em função pura coberta por testes. |
| Como garante que não quebrou nada?                      | Testes unitários nas regras, integração nas políticas de acesso, E2E no fluxo principal, todos obrigatórios no CI.                    |

## 6. Armadilhas comuns

| ARMADILHA                           | POR QUE ATRAPALHA E O QUE FAZER                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Empilhar telas em vez de aprofundar | Vinte telas rasas valem menos que cinco com regra de negócio real. Prefira profundidade em um fluxo a largura em muitos.  |
| Deixar autorização só no front-end  | Esconder o botão não protege o dado. Qualquer chamada direta à API expõe tudo. A política precisa estar no banco.         |
| Adiar os testes para o final        | Testes escritos depois viram teatro de cobertura. Escreva junto com a regra que eles protegem.                            |
| Um único commit gigante             | O histórico é parte do portfólio: mostra como você decompõe problema. Commits pequenos e descritivos, sempre.             |
| README sem imagem e sem link        | Se o avaliador precisa clonar para ver algo, ele não vê. Captura de tela no topo e link de demonstração são inegociáveis. |
| Trocar de stack no meio do caminho  | Projeto terminado vale mais que projeto moderno inacabado. Só troque com motivo escrito em uma ADR.                       |
| Dados de demonstração genéricos     | "Teste 1, Teste 2" destrói a credibilidade do domínio. O seed precisa parecer um sistema em uso real.                     |
| Perseguir 100% de cobertura         | Consome tempo que renderia mais em funcionalidade ou segurança. Cubra as regras críticas e declare a meta honestamente.   |

### Prioridade se o tempo for curto

Havendo tempo para apenas três frentes, escolha nesta ordem: **Fase 1** (dados e segurança), **Fase 2** (CI e deploy público) e **Fase 6** (apresentação). Juntas, elas respondem cinco das seis perguntas do capítulo 1 — e são exatamente as que um projeto de portfólio comum deixa em branco.